FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

IT0049
(WEB SYSTEM TECHNOLOGIES)

TECHNICAL FORMATIVE ASSESSMENT

7

MAKING IT EDITABLE IN LARAVEL

Student Name / Group Name:	QUEJADA, RAINA KISHAN S.
Members (if Group):	**Name** / **Role**
Section:	TW35
Professor:	Mr. Ric Cristian A. Frias

PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design, implement and evaluate computer-based systems or applications to meet desired needs and requirements. [PO: C]

COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- Apply advanced web development principles in creating, debugging, validating, and securing database-backed web applications. [CLO: 2]

INTENDED LEARNING OUTCOME/S (ILO) OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Build and validate Laravel forms for creating new records.
- Redisplay a form with validation errors and the user's old input.
- Implement an edit and update workflow for existing records.
- Accept, validate, prepare, and store an uploaded image file using Laravel's Storage facade.

BACKGROUND INFORMATION

This activity replicates TFA3 in Laravel. Laravel's request validation rejects bad input using the same kind of rules as CodeIgniter (required, valid email, and so on) and automatically redisplays the form with old() input and $errors when validation fails. File uploads are handled through the Storage facade, with an instructor-approved image package used when resizing or cropping is needed, instead of a manual move() call, but the underlying idea — validate, prepare for display, store safely, save only the filename — is unchanged.

GRADING SYSTEM / RUBRIC

CRITERIA	Description
Functionality & Requirements Completeness (40 points max)	All required pages and features are present and work as specified in the activity instructions.
Code Structure & Organization (25 points max)	Controllers, models, and views are separated correctly and follow the framework's conventions; code is readable and sensibly named.
Form Validation & File Handling (20 points max)	Forms reject invalid input with clear error messages and preserve old() input on redisplay; the uploaded file is validated by type and size, prepared for display, stored on the public disk, and only its path is saved to the database.
Documentation & Submission Quality (15 points max)	The GitHub repository is organized and complete, the README explains how to set up and run the project, and the hosted link is live and matches the submitted code.

LABORATORY ACTIVITY

Extend the Laravel POS application from TFA6 with the ability to create and edit customer and user accounts, and add avatar upload with display-ready image preparation for user accounts.

1. Add a Create Customer form (/customers/create) that validates required fields (full_name required, email required and valid) before inserting a record.
2. Add a Create User form (/users/create) that validates required fields (username required and unique, full_name required) before inserting a record.

3. Add edit pages for both customer and user accounts that pre-fill the form with the existing record's data and update it on submit.
4. Add an avatar column to the users table via a migration. On the user edit form, add a file input for a profile picture; validate that the uploaded file is an image no larger than 2MB, prepare a display-ready copy or thumbnail with an instructor-approved image package, store it using Storage::disk('public'), and save only the resulting path in the avatar column.
5. Display each user's prepared avatar or thumbnail on the User Accounts listing page (fall back to a placeholder image if none was uploaded).

6. CONTROLLERS

CustomerController.php

app > Http > Controllers > CustomerController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Customer;
use Illuminate\Http\Request;

class CustomerController extends Controller
{
    public function index() {
        return view('customers.index', ['customers' => Customer::all()]);
    }

    public function create() {
        return view('customers.create');
    }

    public function store(Request $request) {
        $validated = $request->validate([
            'full_name' => ['required', 'string'],
            'email'     => ['required', 'email'],
        ]);

        Customer::create($validated);
        return redirect()->route('customers.index')->with('success', 'Customer created!');
    }

    public function edit($id) {
        $customer = Customer::findOrFail($id);
        return view('customers.edit', compact('customer'));
    }

    public function update(Request $request, $id) {
        $customer = Customer::findOrFail($id);

        $validated = $request->validate([
            'full_name' => ['required', 'string'],
            'email'     => ['required', 'email'],
        ]);

        $customer->update($validated);
        return redirect()->route('customers.index')->with('success', 'Customer updated!');
    }
}
```

UserController.php

app > Http > Controllers > UserController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;

class UserController extends Controller
{
    public function index()
    {
        return view('users.index', ['users' => User::all()]);
    }

    public function create()
    {
        return view('users.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'username'  => ['required', 'string', 'unique:users,username'],
            'full_name' => ['required', 'string'],
            'email'     => ['required', 'email', 'unique:users,email'],
            'avatar'    => ['nullable', 'image', 'max:2048'],
        ]);

        $validated['name'] = $validated['full_name'];

        $validated['password'] = Str::random(16);

        if ($request->hasFile('avatar')) {
            $path = $request->file('avatar')->store('avatars', 'public');

            $validated['avatar'] = $path;
        }

        User::create($validated);

        return redirect()->route('users.index')
            ->with('success', 'User created successfully!');
    }

    public function edit($id)
    {
        $user = User::findOrFail($id);

        return view('users.edit', compact('user'));
    }

    public function update(Request $request, $id)
    {
        $user = User::findOrFail($id);

        $validated = $request->validate([
            'username'  => ['required', 'string', 'unique:users,username,' . $id],
            'full_name' => ['required', 'string'],
            'avatar'    => ['nullable', 'image', 'max:2048'],
        ]);

        $validated['name'] = $validated['full_name'];

        if ($request->hasFile('avatar')) {

            if ($user->avatar) {
                Storage::disk('public')->delete($user->avatar);
```

```
70              }
71
72              $path = $request->file('avatar')->store('avatars', 'public');
73
74              $validated['avatar'] = $path;
75          }
76
77          $user->update($validated);
78
79          return redirect()->route('users.index')
80              ->with('success', 'User profile updated successfully!');
81      }
82  }
```

MODELS

Customer.php

app > Models > Customer.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Customer extends Model
{
    use HasFactory;

    protected $fillable = ['full_name', 'email'];
}
```

User.php

app > Models > User.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;

class User extends Authenticatable
{
    use HasFactory;

    protected $fillable = [
        'name',
        'username',
        'full_name',
        'email',
        'password',
        'avatar',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
```

```php
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }
}
```

DATABASE (migrations)

2026_10_31_042654_create_tasks_table.php

database > migrations > 2026_10_31_042654_create_tasks_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{

    public function up(): void
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->string('status')->default('pending');
        $table->timestamps();
    });
}


    public function down(): void
    {
        Schema::dropIfExists('tasks');
    }
};
```

2026_11_10_025613_add_avatar_to_users_table.php

database > migrations > 2026_11_10_025613_add_avatar_to_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        if (!Schema::hasColumn('users', 'avatar')) {
            Schema::table('users', function (Blueprint $table) {
                $table->string('avatar')->nullable();
            });
        }
    }

    public function down(): void
    {
        if (Schema::hasColumn('users', 'avatar')) {
            Schema::table('users', function (Blueprint $table) {
                $table->dropColumn('avatar');
            });
        }
    }
};
```

2026_11_10_032355_add_username_to_users_table.php

database > migrations > 2026_11_10_032355_add_username_to_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        if (!Schema::hasColumn('users', 'username')) {
            Schema::table('users', function (Blueprint $table) {
                $table->string('username')->unique();
            });
        }

        if (!Schema::hasColumn('users', 'full_name')) {
            Schema::table('users', function (Blueprint $table) {
                $table->string('full_name');
            });
        }
    }

    public function down(): void
    {
        if (Schema::hasColumn('users', 'username')) {
            Schema::table('users', function (Blueprint $table) {
                $table->dropColumn('username');
            });
        }

        if (Schema::hasColumn('users', 'full_name')) {
            Schema::table('users', function (Blueprint $table) {
                $table->dropColumn('full_name');
            });
        }
    }
};
```

2026_11_11_160344_create_customers_table.php

database > migrations > 2026_11_11_160344_create_customers_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('customers', function (Blueprint $table) {
            $table->id();
            $table->string('full_name');
            $table->string('email')->unique();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('customers');
    }
};
```

ROUTES

web.php

routes > web.php

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\CustomerController;
use App\Http\Controllers\UserController;

Route::get('/customers', [CustomerController::class, 'index'])->name('customers.index');
Route::get('/customers/create', [CustomerController::class, 'create'])->name('customers.create');
Route::post('/customers', [CustomerController::class, 'store'])->name('customers.store');
Route::get('/customers/{id}/edit', [CustomerController::class, 'edit'])->name('customers.edit');
Route::put('/customers/{id}', [CustomerController::class, 'update'])->name('customers.update');

Route::get('/users', [UserController::class, 'index'])->name('users.index');
Route::get('/users/create', [UserController::class, 'create'])->name('users.create');
Route::post('/users', [UserController::class, 'store'])->name('users.store');
Route::get('/users/{id}/edit', [UserController::class, 'edit'])->name('users.edit');
Route::put('/users/{id}', [UserController::class, 'update'])->name('users.update');
```

Submission Requirements:

- A link to a GitHub repository containing your raw project files, including your database export or migration and seeder files.
- A link to your hosted, working version of the application.

REFERENCES

- Laravel Documentation. Validation. https://laravel.com/docs/13.x/validation
- Laravel Documentation. File Storage. https://laravel.com/docs/13.x/filesystem